MODÉLISATION DES MENACES

Analyse des Menaces

APPLICATION	ENVIRONNEMENT	AUTEUR	MENACES IDENTIFIÉES
un assistant interne IT	Production	leila saddad	32

01 Description de l'Application

NOM	VERSION	STACK TECHNIQUE
un assistant interne IT	**v1.0**	**FastAPI / Mistral / Gemini / pytm / WeasyPrint**

L'application est un assistant IT interne de type web (React) accessible uniquement aux employés via navigateur, conçue en architecture N-Tier avec microservices (Spring Boot) communiquant en REST interne. Elle s'authentifie via LDAP directement auprès d'Active Directory pour valider les utilisateurs internes et traite des données sensibles (documents internes) sans upload ni échange d'emails. Chaque microservice possède sa propre base de données relationnelle PostgreSQL hébergée sur AWS RDS, avec une instance dédiée à pgvector pour stocker les embeddings locaux (modèle RAG) et indexer la base de connaissances interne. Le chatbot intègre Azure OpenAI via API externe, interrogeant dynamiquement la base vectorielle pour générer des réponses contextuelles sans appeler d'outils externes ni exposer directement le LLM. Les flux critiques incluent l'authentification LDAP vers AD, les requêtes REST entre microservices, l'accès aux données sensibles dans RDS, et les appels API vers Azure OpenAI, le tout dans un environnement cloud AWS sans broker ni tâches asynchrones. La criticité fonctionnelle moyenne et l'absence d'agents ou de ML hors LLM/RAG simplifient le périmètre, mais les interactions avec AD, le LLM externe et les données sensibles nécessitent une attention particulière pour le threat modeling.

02 Diagramme de Flux de Données

un assistant interne IT — DFD

NIVEAU 0

Niveau 0

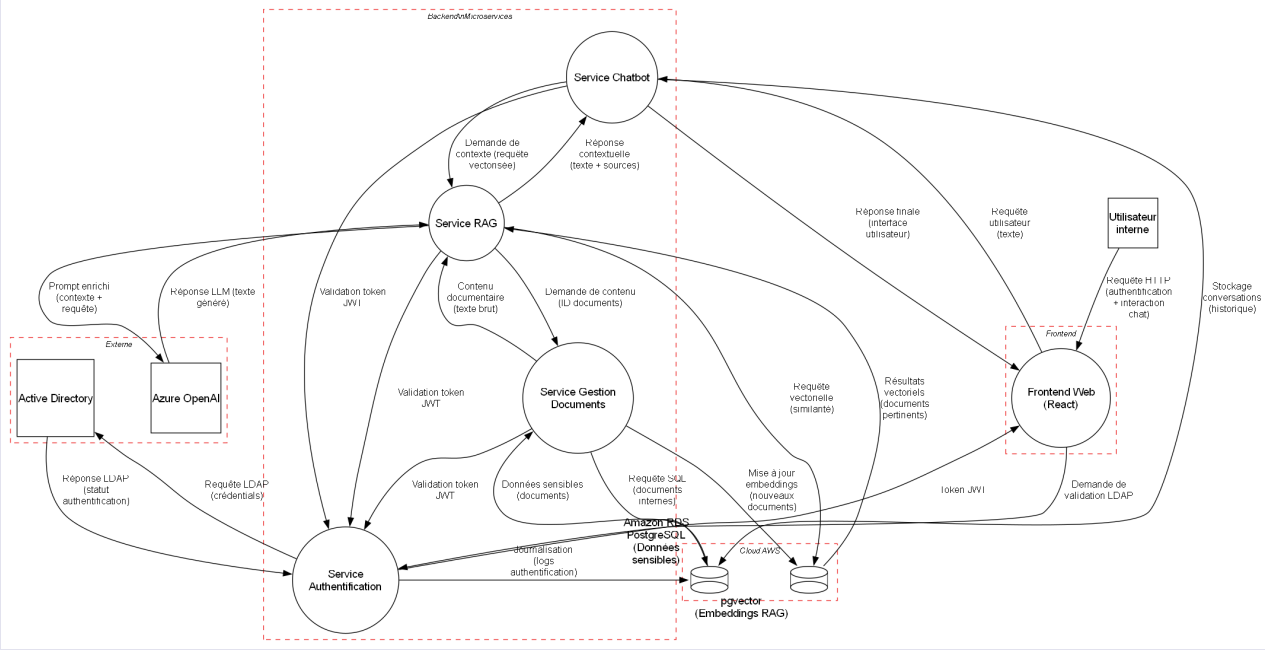

Figure 1 - Diagramme de flux de donnees et frontieres de confiance.

DFD-01

03 Menaces Retenues du Catalogue

Menaces issues du catalogue et retenues selon le contexte technique et fonctionnel de l application.

MENACE	DESCRIPTION
SQL injection	Dans l'application d'assistant IT interne, un attaquant injecte des requêtes SQL malveillantes dans une base de données PostgreSQL (y compris pgvector pour les embeddings RAG) via des entrées utilisateur non sécurisées, comme les champs de recherche ou les paramètres de requêtes API.
AI supply chain tampering	Dans l'assistant IT interne, un attaquant compromet un composant de la chaîne d’approvisionnement de l'IA, comme les documents internes bruts, les pipelines de génération d'embeddings, ou même la base vectorielle pgvector elle-même, afin d’introduire un comportement malveillant dans la logique RAG.

MENACE	DESCRIPTION
Broken Authentication	Les mécanismes d'authentification de l'assistant IT interne, notamment l'authentification LDAP directe auprès d'Active Directory et la gestion des sessions pour l'accès aux microservices via l'interface web (React), présentent des failles (brute-force, jeton faible, absence de ré-authentification) permettant la prise de contrôle de comptes d'employés.
Broken Object Level Authorization	L'API REST des microservices de l'assistant IT interne n'applique pas un contrôle de propriété adéquat sur chaque ressource (documents internes, données utilisateurs) accédée. Ainsi, un employé malveillant ou un attaquant ayant compromis un compte pourrait accéder aux objets (documents, informations) d'autrui en substituant l'identifiant dans la requête API.
Broken Object Property Level Authorization	L'API REST des microservices (Spring Boot) de l'assistant IT interne expose des propriétés d'objets (ex: champs sensibles de documents internes ou d'informations utilisateur) dans ses réponses, ou accepte leur modification sans contrôle granulaire au niveau du champ, même si l'utilisateur n'est pas autorisé à voir ou modifier ces propriétés.
Bruteforce	Un attaquant tente d’accéder aux comptes des employés de l'assistant IT interne en testant systématiquement toutes les combinaisons possibles d'identifiants et de mots de passe sur l'interface de connexion web, ou en ciblant les APIs d'authentification LDAP vers Active Directory.
Cross-Site Request Forgery	Un attaquant tente de forcer un utilisateur (employé) authentifié sur l'assistant IT interne à exécuter une action non désirée, à son insu, en exploitant les vulnérabilités du frontend React ou des APIs REST des microservices.
Cross-Site Scripting (XSS)	Un attaquant injecte du code JavaScript malveillant dans l'application web React de l'assistant IT via des entrées utilisateur non filtrées (par exemple, champs de recherche, noms de documents, ou même les réponses du LLM si elles sont mal gérées) affichées à d'autres employés.
Data Poisoning	Un attaquant manipule intentionnellement les documents internes ou les données utilisées pour construire la base de connaissances et les embeddings stockés dans pgvector, afin d’altérer le comportement, les performances ou les décisions du modèle RAG/LLM de l'assistant IT.
deserialization injection	Un attaquant exploite la désérialisation de données non fiables (par exemple, dans les communications REST entre microservices Spring Boot) pour manipuler des objets applicatifs ou exécuter du code malveillant sur les serveurs des microservices de l'assistant IT interne.

MENACE	DESCRIPTION
Directory Traversal	Dans l'application d'assistant IT interne, un attaquant manipule les chemins de fichiers (par exemple, dans les requêtes d'accès aux 'documents internes' pour le RAG) pour accéder à des fichiers ou répertoires en dehors du répertoire autorisé sur les serveurs des microservices ou les systèmes de stockage d'AWS.
Direct prompt injection	Un attaquant injecte des instructions malveillantes directement dans les prompts du chatbot de l'assistant IT interne pour manipuler le comportement du modèle Azure OpenAI, le pousser à ignorer ses directives ou à divulguer des informations sensibles.
Distributed Denial of Service	Un attaquant submerge les serveurs des microservices (Spring Boot) et le frontend (React) de l'assistant IT interne avec un grand volume de trafic provenant de multiples sources, rendant l'application indisponible pour les employés. Le backend des microservices peut également être ciblé.
File Inclusion	Un attaquant exploite les microservices de l'assistant IT interne pour inclure et exécuter des fichiers locaux ou distants via des entrées utilisateur non sécurisées, potentiellement dans le contexte de la gestion des 'documents internes' pour le RAG.
File Processing Injection	L'assistant IT interne traite des 'documents internes' pour sa base de connaissances RAG. Un attaquant injecte des données malveillantes dans ces fichiers afin d’exécuter du code, exfiltrer des données ou compromettre le système lors de leur traitement (parsing, indexation, etc.) par les microservices.
HTTP Request Smuggling	Dans l'architecture N-Tier de l'assistant IT interne avec microservices (Spring Boot) communiquant en REST, un attaquant exploite une désynchronisation entre un proxy frontal (load balancer, WAF) et les microservices backend pour injecter une requête HTTP cachée.
Indirect prompt injection	Un attaquant injecte des instructions malveillantes dans les 'documents internes' qui constituent la base de connaissances du système RAG. Le modèle Azure OpenAI de l'assistant IT interne, en interrogeant cette base vectorielle, lit et exécute indirectement ces instructions sans que l'utilisateur n'ait à les saisir directement.
Information Disclosure	L'application d'assistant IT interne traite des données sensibles (documents internes, informations d'employés) et est hébergée sur AWS. Une mauvaise configuration du système, des messages d'erreur détaillés, ou des logs non sécurisés pourraient exposer ces informations à des employés non autorisés ou même à des attaquants externes.

MENACE	DESCRIPTION
Jwt manipulation	Les microservices (Spring Boot) de l'assistant IT interne communiquent en REST. Si des JSON Web Tokens (JWT) sont utilisés pour l'authentification et l'autorisation entre ces services, un attaquant pourrait manipuler ces tokens pour usurper l'identité de services légitimes, obtenir des privilèges non autorisés ou accéder à des données sensibles.
LDAP injection	L'application d'assistant IT interne s'authentifie directement via LDAP auprès d'Active Directory. Un attaquant injecte des requêtes malveillantes dans le filtre LDAP via des entrées utilisateur non sécurisées (par exemple, dans le champ d'identifiant ou de mot de passe) afin de manipuler les requêtes d'authentification ou de recherche.
Long-term Memory Poisoning	L'assistant IT interne utilise une base vectorielle pgvector pour stocker les embeddings locaux de sa base de connaissances interne (documents internes) comme mémoire persistante pour le modèle RAG. Un attaquant injecte ou altère des informations malveillantes dans cette mémoire persistante.
Man-in-the-Middle	Un attaquant intercepte et potentiellement modifie les communications entre les composants de l'application d'assistant IT interne : Frontend React et Microservices, Microservices entre eux (REST interne), ou Microservices et Active Directory (LDAP), et Microservices et Azure OpenAI (API externe).
ML model evasion	Un attaquant manipule les entrées (prompts) du modèle Azure OpenAI du chatbot de l'assistant IT interne afin de contourner ses prédictions ou décisions lors de l'inférence, le poussant à générer des réponses indésirables ou à ignorer des instructions.
Model denial of service	Un attaquant exploite les limites computationnelles, opérationnelles ou économiques du modèle Azure OpenAI (via son API) ou de la couche RAG (base pgvector, microservices) de l'assistant IT interne en générant une charge excessive ou anormale.
Model stealing	L'application utilise une logique RAG avec des embeddings locaux stockés dans pgvector pour sa base de connaissances interne. Un attaquant tente d'accéder, copier, extraire ou reconstruire ces embeddings ou la logique RAG sous-jacente sans autorisation, via des failles d'infrastructure, des fuites internes ou des requêtes API, afin de recréer la base de connaissances ou exploiter cette propriété intellectuelle.
Model Tampering	L'application d'assistant IT interne utilise une base vectorielle pgvector pour les embeddings de sa base de connaissances. Un attaquant tente de compromettre l'intégrité de cette base de connaissances en modifiant ses embeddings, ses artefacts (documents sources) ou son processus de mise à jour.

MENACE	DESCRIPTION
Password Reset takeover	Bien que l'authentification principale se fasse via LDAP/Active Directory, si l'application d'assistant IT interne gère un processus de réinitialisation de mot de passe (par exemple, pour un compte de service local ou des fonctionnalités spécifiques), un attaquant pourrait exploiter des failles dans ce mécanisme pour prendre le contrôle du compte d'un employé sans disposer de ses identifiants Active Directory.
Race Condition	Dans l'architecture microservices (Spring Boot) de l'assistant IT interne, avec des accès concurrents aux bases de données PostgreSQL et à d'autres ressources partagées, un attaquant exploite un problème de concurrence où plusieurs requêtes modifient une ressource en même temps.
Server-Side Request Forgery (SSRF)	Dans l'assistant IT interne, un attaquant exploite une mauvaise validation des entrées (par exemple, dans les requêtes de recherche RAG ou les appels à Azure OpenAI) pour forcer un microservice Spring Boot à envoyer des requêtes vers des ressources internes (AWS metadata, autres services internes) ou externes non autorisées avec ses propres privilèges.
Session Fixation	Un attaquant tente de forcer un employé de l'assistant IT interne à utiliser un identifiant de session connu par l'attaquant afin de prendre le contrôle de sa session après que l'employé se soit authentifié via LDAP/Active Directory sur l'application web React.
Short-term Memory Poisoning	Dans le contexte du chatbot de l'assistant IT interne, un attaquant injecte ou altère des informations malveillantes dans la mémoire temporaire de l'agent (le contexte de la conversation envoyé au LLM Azure OpenAI) afin d'influencer son raisonnement ou ses décisions pendant la session en cours.
User Impersonation	Dans le contexte du chatbot de l'assistant IT interne, un attaquant vise à usurper l'identité d'un employé légitime pour faire exécuter des actions par le chatbot (via Azure OpenAI/RAG) en son nom, potentiellement pour accéder à des données sensibles ou manipuler les réponses.

04 Scenarios d Attaque

Scenarios d attaque selectionnes selon l architecture et les composants identifies.

MENACE	SCENARIO
SQL injection	Un attaquant injecte du code SQL dans un formulaire de connexion pour se connecter sans identifiants valides.
SQL injection	Un attaquant exploite une vulnérabilité SQL Injection pour extraire des informations sensibles de la base de données PostgreSQL, y compris des documents internes et des embeddings vectoriels.
SQL injection	Un attaquant injecte des commandes SQL pour modifier ou supprimer des données dans la base, altérant la base de connaissances ou d'autres données sensibles.
AI supply chain tampering	Un attaquant injecte des données malveillantes dans un document interne qui sera utilisé pour créer des embeddings de la base de connaissances.
AI supply chain tampering	Un attaquant injecte un adaptateur ou un script malveillant dans un pipeline de traitement de documents pour modifier la génération d'embeddings sans altérer le modèle principal.
AI supply chain tampering	Un attaquant falsifie les informations d’origine d’un document interne afin de masquer sa provenance réelle et tromper le système RAG.
Broken Authentication	L’attaquant cible les endpoints d’authentification LDAP ou les APIs de l’application, soumettant massivement des combinaisons d'identifiants/mots de passe pour compromettre l'authentification.
Broken Authentication	L’attaquant s’appuie sur des mots de passe faibles ou réutilisés, ou des secrets de signature faibles pour les tokens d'accès internes, pour s'authentifier auprès de l'application ou des microservices.
Broken Authentication	L’attaquant récupère un jeton d’authentification interne (ex: JWT) entre microservices, le modifie (signature, claims), puis le rejoue pour faire accepter un jeton forgé ou altéré.
Broken Object Level Authorization	L'attaquant intercepte une requête API valide vers une ressource qui lui est accessible, puis remplace l'identifiant de l'objet ciblé par d'autres valeurs plausibles (ex: ID séquentiels ou UUID devinables).
Broken Object Level Authorization	L'attaquant formule des requêtes vers un microservice de données en substituant des références d'objets appartenant à un domaine métier différent ou à un autre utilisateur, et le microservice répond sans vérifier la légitimité.

MENACE	SCENARIO
Broken Object Property Level Authorization	L'attaquant observe les réponses API d'opérations légitimes et constate que des propriétés sensibles d'objet (ex: statut interne d'un document) sont retournées sans être nécessaires à la fonction demandée.
Broken Object Property Level Authorization	L'attaquant observe le schéma des objets retournés par l'API, en déduit les propriétés internes de l'entité côté serveur, puis les soumet dans le corps d'une requête de modification (mass assignment).
Bruteforce	Un attaquant teste plusieurs mots de passe sur une interface de login pour trouver le bon.
Bruteforce	Un attaquant utilise une base de données d'identifiants volés lors de fuites précédentes (credential stuffing) pour tester massivement ces combinaisons sur l'application.
Bruteforce	Un attaquant lance des tentatives massives sur l'interface LDAP ou les APIs REST d'authentification pour obtenir un accès.
Cross-Site Request Forgery	Un attaquant pousse la victime (employé) à cliquer sur un lien/pièce jointe malveillant qui déclenche une action sur l'application.
Cross-Site Request Forgery	Un attaquant force une requête pour modifier les préférences de l'utilisateur ou des informations relatives aux documents internes.
Cross-Site Request Forgery	Un attaquant force la victime à envoyer une requête critique (ex: modification de permissions) sans sa connaissance.
Cross-Site Scripting (XSS)	Un attaquant injecte un script pour récupérer les cookies de session d’un utilisateur authentifié.
Cross-Site Scripting (XSS)	Un attaquant injecte du JavaScript pour modifier la page affichée à l'employé et le piéger.
Cross-Site Scripting (XSS)	Un attaquant injecte un faux formulaire dans la page pour voler des identifiants ou des données.
Data Poisoning	Un attaquant injecte des données incorrectes ou des informations malveillantes dans les documents internes utilisés pour l'entraînement RAG, afin de réduire la précision ou l'exactitude du modèle.

MENACE	SCENARIO
Data Poisoning	Un attaquant modifie les données dans la base vectorielle pgvector pour influencer les décisions du modèle dans un cas précis, par exemple pour le faire divulguer des informations sensibles sous certaines requêtes.
Data Poisoning	Un attaquant injecte des données spécifiques (triggers) pour créer un comportement caché dans le modèle RAG, activé par une requête particulière de l'utilisateur.
deserialization injection	Un attaquant modifie un objet sérialisé envoyé via REST pour injecter du code exécuté lors de la désérialisation par un microservice Spring Boot.
deserialization injection	Un attaquant modifie les valeurs d'un objet désérialisé pour obtenir des privilèges administratifs au sein de l'application.
deserialization injection	Un attaquant envoie des objets volumineux ou complexes via une API REST pour épuiser les ressources des microservices, provoquant un déni de service.
Directory Traversal	Un attaquant modifie un paramètre de requête pour inclure un fichier du système (ex: /etc/passwd ou fichiers de configuration) et extraire des informations sensibles.
Directory Traversal	Un attaquant accède à des fichiers internes (fichiers de configurations, clés API) en manipulant les chemins dans les requêtes.
Directory Traversal	Un attaquant accède à des fichiers sensibles permettant d'exécuter du code ou modifier le comportement du système.
Direct prompt injection	Un attaquant injecte des instructions dans le prompt pour ignorer les règles de sécurité du modèle Azure OpenAI.
Direct prompt injection	Un attaquant insère un prompt demandant explicitement au modèle de révéler des informations internes ou confidentielles (par exemple, tirées de la base de connaissances RAG).
Direct prompt injection	Un attaquant modifie les instructions du prompt pour influencer les réponses du modèle, le rendant plus enclin à générer des informations biaisées ou des erreurs.
Distributed Denial of Service	Un attaquant envoie un énorme volume de requêtes depuis plusieurs machines pour saturer les ressources de l'application web.
Distributed Denial of Service	Un attaquant envoie des requêtes HTTP légitimes mais massives pour surcharger les microservices Spring Boot.

MENACE	SCENARIO
Distributed Denial of Service	Un attaquant exploite des serveurs tiers pour amplifier le trafic vers l'application.
File Inclusion	Un attaquant manipule un paramètre pour inclure un fichier du système et extraire les informations sensibles.
File Inclusion	Un attaquant force l’application à charger un fichier hébergé sur un serveur externe (potentiellement malveillant).
File Inclusion	Un attaquant combine la faille d'inclusion de fichiers avec d’autres techniques (ex: upload de fichiers si une fonction cachée existe) pour exécuter du code.
File Processing Injection	Un attaquant injecte une formule malveillante dans un document interne (ex: un tableur) qui sera traitée lors de l'indexation, et pourrait être exécutée si le parseur est vulnérable.
File Processing Injection	Un attaquant injecte du code LaTeX dans un document texte pour le RAG, permettant de lire des fichiers ou exécuter des commandes si l'outil de traitement est vulnérable.
File Processing Injection	Un attaquant injecte des liens ou des scripts dans les documents internes, qui seraient activés si le LLM ou le client les interprétait.
HTTP Request Smuggling	Un attaquant injecte une requête HTTP cachée pour contourner les contrôles de sécurité du serveur frontal et accéder à des ressources protégées ou internes aux microservices.
HTTP Request Smuggling	Un attaquant manipule les requêtes HTTP pour interférer avec celles d’autres utilisateurs et récupérer leurs données.
HTTP Request Smuggling	Un attaquant envoie des requêtes malformées pour altérer le traitement des requêtes suivantes par le microservice backend.
Indirect prompt injection	Un attaquant insère des instructions cachées dans un document interne traité par le modèle RAG afin de déclencher des actions malveillantes ou des réponses spécifiques.
Indirect prompt injection	Un attaquant injecte du contenu malveillant dans la base de connaissances vectorielle (pgvector) utilisée par le modèle RAG, influençant les réponses futures.
Indirect prompt injection	Un attaquant manipule des métadonnées ou le formatage des documents internes pour influencer l'interprétation par le modèle, contournant les filtres de prompt.

MENACE	SCENARIO
Information Disclosure	Un attaquant accède à des fichiers de configuration, logs ou backups exposés par inadvertance sur le système de fichiers des microservices ou le stockage AWS.
Information Disclosure	Un attaquant récupère des informations sensibles (ex: identifiants, URLs internes) présentes dans le code source du frontend React, les commentaires HTML ou les scripts JavaScript exposés.
Information Disclosure	Un attaquant exploite des messages d'erreur détaillés pour obtenir des informations sur le système (chemins, base de données, versions de framework).
Information Disclosure	Un attaquant obtient des informations sensibles via des en-têtes HTTP ou bannières système trop verbeuses des microservices Spring Boot.
Jwt manipulation	Un attaquant modifie le header du JWT pour désactiver la signature (ex: alg=none) et forge un token valide.
Jwt manipulation	Un attaquant modifie le payload d'un JWT pour obtenir des rôles administratifs ou des privilèges plus élevés.
Jwt manipulation	Un attaquant exploite une mauvaise gestion des clés de signature JWT pour signer ses propres requêtes malveillantes.
LDAP injection	Un attaquant injecte des caractères LDAP dans un champ de login pour modifier la requête et contourner l'authentification.
LDAP injection	Un attaquant injecte des opérateurs logiques LDAP pour altérer la logique de recherche et accéder à des entrées non autorisées dans l'Active Directory.
LDAP injection	Un attaquant modifie une requête LDAP pour récupérer plus d'informations que prévu sur les utilisateurs ou groupes.
Long-term Memory Poisoning	L'attaquant dépose un document malveillant dans une source indexée par l'agent. Lors d'une recherche de contexte, l'agent récupère ce document et l'intègre à son traitement comme une donnée de référence, influençant les réponses futures.
Long-term Memory Poisoning	L'attaquant exploite un défaut d'authentification ou d'autorisation sur l'API de mémoire ou directement sur pgvector. Il accède aux enregistrements stockés puis modifie les vecteurs, les métadonnées ou les entrées de mémoire à la source.

MENACE	SCENARIO
Long-term Memory Poisoning	L’attaquant injecte un contenu malveillant qui est intégré plus tard lors d’un processus de consolidation, de re-embedding, ou de summarization périodique des documents internes.
Man-in-the-Middle	Un attaquant intercepte le trafic entre le frontend et le backend, ou entre microservices, pour lire les informations échangées (données sensibles, tokens).
Man-in-the-Middle	Un attaquant se fait passer pour un microservice légitime ou l'Active Directory en présentant un faux certificat.
Man-in-the-Middle	Un attaquant intercepte et modifie les données échangées (prompts, réponses LLM, requêtes LDAP) pour manipuler le comportement de l'application ou exfiltrer des données.
ML model evasion	Un attaquant fournit des prompts conçus pour être en dehors du domaine d’entraînement habituel afin de provoquer des erreurs de classification ou des réponses inattendues.
ML model evasion	Un attaquant manipule un prompt pour que le modèle génère une réponse spécifique choisie par l'attaquant.
ML model evasion	Un attaquant exploite les réponses du modèle (même indirectement) pour affiner ses prompts et améliorer l’évasion.
Model denial of service	Un attaquant envoie un grand nombre de requêtes simultanées pour remplir la file d’attente du modèle Azure OpenAI ou des microservices RAG.
Model denial of service	Un attaquant envoie des prompts dépassant la taille maximale du contexte ou des requêtes anormalement coûteuses pour maximiser la consommation des ressources.
Model denial of service	Un attaquant soumet des entrées spécialement conçues pour maximiser le temps de calcul du modèle ou de la recherche RAG.
Model stealing	Un attaquant envoie un grand nombre de requêtes au modèle RAG pour apprendre progressivement son comportement et extraire les embeddings locaux.
Model stealing	Un attaquant analyse les réponses du modèle RAG pour déduire ses hyperparamètres ou la structure de la base de connaissances.

MENACE	SCENARIO
Model stealing	Un attaquant accède directement à la base pgvector (fichier, stockage, mémoire) pour copier les embeddings.
Model stealing	Un attaquant exploite les scores ou réponses du modèle pour reconstruire des données sensibles utilisées lors de la construction de la base de connaissances.
Model Tampering	Un attaquant modifie directement les embeddings stockés dans pgvector ou les documents sources de la base de connaissances.
Model Tampering	Un attaquant injecte une modification dans la base de connaissances (par exemple, un document contenant un trigger) pour qu'elle réagisse de manière spécifique et malveillante à un prompt caché.
Model Tampering	Un attaquant force le retour vers une ancienne version vulnérable des documents ou embeddings de la base de connaissances.
Password Reset takeover	Un attaquant génère ou devine un token de réinitialisation car celui-ci est court, non aléatoire ou basé sur des valeurs prévisibles.
Password Reset takeover	Un attaquant modifie la requête de reset (ex: en-tête Host) pour que le lien de réinitialisation soit envoyé vers un domaine contrôlé par lui.
Password Reset takeover	Un attaquant envoie directement la requête de changement de mot de passe sans que le token ne soit correctement vérifié côté serveur.
Race Condition	Un attaquant envoie plusieurs requêtes simultanées pour modifier une ressource (ex: un document interne, une préférence utilisateur) avant que le système ne soit mis à jour correctement.
Race Condition	Un attaquant exploite un décalage entre vérification et exécution (TOCTOU - Time-of-Check to Time-of-Use) pour accéder à une ressource protégée.
Race Condition	Un attaquant déclenche des opérations concurrentes provoquant des états incohérents ou des erreurs critiques dans la base de données ou l'état de l'application.
Server-Side Request Forgery (SSRF)	Un attaquant utilise le microservice pour envoyer des requêtes vers d'autres systèmes internes du réseau AWS.
Server-Side Request Forgery (SSRF)	Un attaquant cible les endpoints de métadonnées cloud d'AWS (169.254.169.254) pour obtenir des informations sensibles sur l'instance EC2 du microservice.

MENACE	SCENARIO
Server-Side Request Forgery (SSRF)	Un attaquant force le microservice à appeler des services internes (localhost, réseau privé) non exposés publiquement.
Session Fixation	Un attaquant impose un ID de session à la victime (via URL, cookie), puis attend qu'elle se connecte.
Session Fixation	Un attaquant envoie un lien contenant un ID de session prédéfini à la victime.
Short-term Memory Poisoning	L'attaquant injecte du contenu malveillant dans les segments de la mémoire de travail de l'agent, comme le scratchpad ou les résultats d'outils, via des tours de conversation.
Short-term Memory Poisoning	L'attaquant insère des instructions persistantes ou des faits falsifiés dans une conversation longue pour forcer leur inclusion lors de la phase de résumé (compression) du contexte envoyé au LLM.
User Impersonation	L'attaquant obtient les identifiants ou facteurs d'authentification d'un utilisateur légitime (par phishing ou autre), puis se connecte à l'assistant IT et interagit avec le chatbot comme s'il était cet utilisateur.
User Impersonation	L'attaquant récupère un cookie ou jeton de session associé à une session déjà authentifiée d'un employé, puis le réutilise pour interagir avec le chatbot.
User Impersonation	L'attaquant injecte dans le contexte du chatbot une fausse assertion d'identité, de rôle ou d'autorité utilisateur dans un prompt, ou un message inter-agent, et amène le chatbot à traiter cette déclaration comme valide.

05 Mitigations

MITIGATION À APPLIQUER
Utiliser exclusivement des requêtes préparées (prepared statements) pour toutes les interactions avec la base de données PostgreSQL.
Interdire la concaténation dynamique de chaînes SQL dans le code des microservices Spring Boot.
Valider strictement toutes les entrées utilisateur côté serveur (type, format, taille et contenu).

MITIGATION À APPLIQUER
Chiffrer les données sensibles au repos dans PostgreSQL RDS et en transit via TLS 1.3.
Appliquer le principe du moindre privilège aux comptes d'accès à la base de données et aux rôles IAM sur AWS RDS.
Implémenter une solution de Database Firewall ou un filtrage SQL au niveau du réseau pour bloquer les requêtes malveillantes.
Masquer les erreurs SQL détaillées côté utilisateur pour éviter la divulgation d'informations.
Activer les logs natifs d'AWS RDS pour la base de données et centraliser les traces dans un SIEM.
Restreindre l’accès réseau aux registres de packages, artefacts et modèles aux services internes autorisés uniquement (ex: registres d'images Docker, dépôts de code).
Implémenter un IAM centralisé pour l’accès aux dépôts de code, registries et pipelines CI/CD des microservices.
Appliquer le principe du moindre privilège aux comptes accédant aux registres d’artefacts et aux ressources ML (comme la base vectorielle pgvector).
Vérifier l’intégrité cryptographique des artefacts (documents, embeddings, scripts RAG) avant déploiement ou intégration.
Maintenir un AI Bill of Materials (AI-BOM) pour les modèles, datasets, dépendances et adaptateurs utilisés dans la chaîne RAG.
Scanner les images de conteneurs utilisées dans les pipelines ML et les microservices pour détecter les vulnérabilités.
Appliquer un rate limiting strict par compte, IP, session et endpoint d’authentification (LDAP, API).
Déclencher un account lockout temporaire ou un step-up MFA après N échecs d'authentification.
Imposer un MFA robuste (phishing-resistant si possible) sur les flux de login et de réinitialisation de mot de passe.
Détecter et bloquer les campagnes de credential stuffing à l’aide d’une analyse comportementale.
Générer tous les tokens et session IDs avec un CSPRNG (Cryptographically Secure Pseudo-Random Number Generator) et exiger une entropie suffisante.

MITIGATION À APPLIQUER
Vérifier systématiquement la signature cryptographique de tout token d'authentification (ex: JWT) et valider les claims critiques côté serveur.
Protéger, faire tourner et cloisonner les clés de signature dans un stockage sécurisé (ex: AWS KMS, Secrets Manager).
Vérifier côté serveur que l'identifiant de ressource appartient à l'utilisateur du jeton JWT (ou de la session authentifiée), à chaque appel API.
Ne jamais extraire l'identité de l'appelant depuis la requête elle-même ; la lire uniquement depuis le jeton signé ou la session sécurisée.
Générer les identifiants de ressources en UUID v4 aléatoires pour empêcher l'énumération prévisible.
Utiliser des ACL (Access Control Lists) ou des règles d’accès fines lorsque les permissions doivent être gérées par utilisateur, groupe ou ressource.
Appliquer un contrôle d’accès par rôles (RBAC) ou attributs (ABAC) pour limiter chaque utilisateur aux actions et ressources autorisées selon son rôle.
Isoler les microservices par domaine de données : un service ne peut exposer que ses propres ressources ou celles pour lesquelles il est explicitement autorisé.
Journaliser et alerter sur tout accès à un identifiant n'appartenant pas au contexte de l'appelant.
Définir des DTOs (Data Transfer Objects) distincts par endpoint API, exposant uniquement les propriétés strictement nécessaires dans les réponses.
Éviter les sérialiseurs génériques ; lister explicitement les champs retournés dans chaque réponse.
Valider le payload entrant contre un schéma JSON strict (`additionalProperties: false`) pour interdire la soumission de propriétés non autorisées.
Restreindre la modification aux seules propriétés explicitement autorisées pour le rôle de l'appelant.
Mapper explicitement les champs autorisés vers le modèle métier et désactiver tout binding automatique non maîtrisé par Spring Boot.
Configurer l’API Gateway pour valider le payload entrant contre un schéma JSON strict.

MITIGATION À APPLIQUER
Auditer les réponses API à chaque déploiement pour détecter les propriétés exposées par inadvertance.
Logger et alerter sur toute soumission ou exposition de propriété non autorisée comme événement de sécurité.
Implémenter un rate limiting strict sur les endpoints d’authentification (par IP, compte et device).
Bloquer temporairement le compte après un nombre défini d’échecs de connexion (account lockout).
Implémenter un délai progressif (backoff exponentiel) entre les tentatives de connexion.
Imposer un MFA obligatoire pour tous les comptes d'employés, surtout les plus sensibles.
Implémenter un CAPTCHA adaptatif après détection de comportement suspect sur l'interface web.
Journaliser toutes les tentatives d’authentification (succès/échec) et corréler les événements dans un SIEM.
Appliquer une politique de mots de passe robustes (longueur minimale élevée, complexité) pour les comptes Active Directory.
Déployer un WAF avec protection anti-bot avancée pour le frontend React.
Utiliser des tokens anti-CSRF cryptographiquement forts, uniques par requête et par session, et les valider côté serveur.
Implémenter des cookies de session avec l'attribut `SameSite=Strict` par défaut.
Vérifier l’en-tête `Origin` et le `Referer` des requêtes sur les endpoints critiques.
Exiger une re-authentification forte (step-up authentication) pour toute action critique.
Bloquer toute requête cross-origin non explicitement autorisée via une politique CORS stricte.
Déployer un WAF pour bloquer les requêtes sans référent valide ou avec des patterns CSRF connus.
Implémenter un mécanisme d’échappement systématique de toutes les données en sortie, en fonction du contexte (HTML, JavaScript), dans le frontend React.
Valider et filtrer strictement toutes les entrées utilisateur côté serveur dans les microservices Spring Boot.

MITIGATION À APPLIQUER
Implémenter une politique de sécurité de contenu (CSP) stricte pour restreindre l’exécution de scripts non autorisés.
Déployer un WAF avec des règles de détection et de blocage des attaques XSS.
Activer les attributs de sécurité des cookies (HttpOnly, Secure, SameSite).
Isoler strictement les environnements d'ingestion des documents, de création des embeddings et de production RAG dans des segments réseau distincts.
Appliquer le principe du moindre privilège et un contrôle d'accès strict (RBAC) pour l’accès aux datasets (documents internes) et aux pipelines de génération d'embeddings.
Implémenter un Data Version Control (DVC) pour suivre toute modification des datasets et des embeddings, et vérifier leur intégrité via des hash cryptographiques (SHA-256) avant chaque phase d’entraînement ou d'indexation.
Valider et filtrer toutes les données externes avant ingestion dans la base de connaissances, en supprimant scripts, HTML actif et contenu dynamique.
Mettre en place des mécanismes de quarantaine pour les documents ou embeddings suspects.
Surveiller les performances du modèle après chaque mise à jour de la base de connaissances, et détecter automatiquement les outliers.
Chiffrer les documents internes et les embeddings au repos et en transit.
Éviter l’utilisation de mécanismes de désérialisation natifs non sécurisés (si applicable, bien que Spring Boot utilise généralement Jackson pour JSON qui est plus sûr).
Valider et nettoyer les données sérialisées provenant de sources non fiables avant toute désérialisation.
Implémenter une allowlist stricte des classes autorisées à la désérialisation.
Limiter la profondeur et la taille des objets désérialisés pour prévenir les attaques par déni de service.
Signer toutes les données sérialisées (ex: via HMAC) si leur intégrité est critique, et vérifier la signature avant désérialisation.

MITIGATION À APPLIQUER
Désactiver les fonctionnalités dangereuses de désérialisation (auto-type, polymorphisme dynamique) si non strictement nécessaires.
Appliquer le principe du moindre privilège aux microservices effectuant la désérialisation.
Implémenter une validation stricte des chemins de fichiers basée sur une allowlist et un mappage interne (utiliser des identifiants indirects au lieu de chemins directs fournis par l'utilisateur).
Normaliser tous les chemins de fichiers avant de les utiliser pour éliminer les séquences ../ ou autres caractères spéciaux.
Stocker les documents internes sensibles dans des zones non directement accessibles par les microservices via chemins HTTP/API, mais plutôt via un service dédié avec des contrôles d'accès stricts.
Appliquer le principe du moindre privilège sur les accès aux fichiers pour les comptes de service des microservices.
Chiffrer les fichiers sensibles au repos sur le stockage AWS.
Journaliser toutes les tentatives d’accès aux fichiers et surveiller les accès aux fichiers critiques.
Implémenter une LLM Security Gateway ou AI Firewall pour inspecter les prompts avant et après envoi à Azure OpenAI.
Séparer clairement les instructions système (system prompt) des données utilisateur, et stocker le system prompt uniquement côté backend, empêchant toute modification par les utilisateurs.
Filtrer tous les prompts pour détecter et neutraliser les balises HTML, Markdown, caractères spéciaux ou tentatives d'obfuscation.
Déployer des guardrails pour contrôler les entrées utilisateur et les réponses du modèle, bloquant les contenus interdits ou dangereux.
Refuser les requêtes demandant des secrets, politiques internes ou instructions système.
Limiter le contexte conversationnel réutilisé en cas de prompt suspect et réinitialiser ou isoler le contexte après détection d’une tentative d’injection.
Implémenter l’interdiction d’exécuter directement les sorties LLM comme code ou requête, et une validation humaine pour les actions critiques.

MITIGATION À APPLIQUER
Déployer une protection anti-DDoS en amont du réseau (ex: AWS Shield Advanced) et utiliser un CDN (Content Delivery Network) pour absorber le trafic.
Implémenter un rate limiting strict par IP, endpoint et utilisateur sur l'API Gateway et au niveau applicatif.
Déployer un WAF avec protection anti-DDoS applicatif pour les couches Frontend et Backend.
Implémenter un auto-scaling dynamique des microservices et du frontend pour gérer les pics de charge.
Mettre en place des timeouts et circuit breakers entre les microservices pour éviter l'effet domino.
Monitorer en temps réel le trafic, la latence et l'utilisation des ressources pour détecter les attaques.
Implémenter une allowlist stricte des fichiers autorisés à l’inclusion, et utiliser un mapping interne au lieu d’un chemin fourni par l'utilisateur.
Interdire toute inclusion de fichiers distants.
Valider strictement tous les paramètres liés aux fichiers et empêcher l’accès aux fichiers sensibles.
Isoler les microservices dans des environnements conteneurisés sécurisés et appliquer le principe du moindre privilège sur les accès aux fichiers.
Stocker les documents internes et autres fichiers sensibles en dehors des répertoires accessibles publiquement.
Masquer les erreurs contenant des chemins système pour éviter la divulgation d'informations.
Valider strictement toutes les données avant intégration dans un fichier ou avant traitement par les parseurs de documents.
Neutraliser les caractères interprétables par le format cible et appliquer une normalisation des données avant traitement.
Empêcher toute inclusion de ressources externes dans les fichiers traités et bloquer les appels réseau initiés lors de l’ouverture ou du traitement des fichiers.
Interdire les mécanismes d’exécution automatique (scripts PDF, macros) dans les documents traités pour la base de connaissances RAG.

MITIGATION À APPLIQUER
Exécuter les moteurs de traitement de fichiers avec des privilèges minimaux et dans des environnements isolés (sandbox).
S’assurer que le front-end (API Gateway/Load Balancer) et les microservices back-end utilisent la même interprétation des en-têtes HTTP (Content-Length, Transfer-Encoding).
Rejeter toute requête mal formée ou incohérente.
Nettoyer et normaliser les en-têtes HTTP avant traitement.
Ne pas faire confiance aux requêtes en provenance du front-end sans validation, et valider les sessions et tokens indépendamment du proxy frontal.
Mettre à jour et homogénéiser les versions des serveurs HTTP et des composants intermédiaires.
Valider et filtrer toutes les données externes (documents internes) avant ingestion dans la base de connaissances RAG et la base vectorielle pgvector.
Scanner les contenus pour détecter du contenu malveillant ou injecté, et refuser les contenus contenant des instructions destinées au modèle LLM.
Supprimer ou neutraliser les instructions cachées (ex: balises invisibles, texte couleur blanche) dans les documents.
Nettoyer les documents en supprimant scripts, HTML actif et contenu dynamique avant leur indexation.
Implémenter un pipeline de sanitization avant indexation et un scoring de confiance des données externes, refusant les données à faible niveau de confiance.
Filtrer toutes les réponses du LLM via un LLM Firewall ou guardrails pour empêcher la divulgation du system prompt, de données sensibles ou les tentatives d’exfiltration.
Tracer l’origine des données (documents internes) utilisées par le RAG pour chaque réponse.
Déployer un modèle secondaire (LLM-as-a-judge) pour analyser les entrées et sorties afin de détecter les injections indirectes.
Classifier les données sensibles et mettre en place une solution DLP (Data Loss Prevention) pour détecter et bloquer les fuites d'information.

MITIGATION À APPLIQUER
Masquer ou tronquer les données sensibles dans les interfaces utilisateur et les journaux.
Éviter l’exposition de secrets, clés (API Azure OpenAI), mots de passe et tokens, et les stocker dans un gestionnaire de secrets sécurisé (ex: AWS Secrets Manager).
Désactiver les messages d’erreur détaillés en production et supprimer les fichiers de debug, logs et backups accessibles publiquement.
Durcir les serveurs web et applicatifs (microservices Spring Boot) selon les benchmarks de sécurité.
Sécuriser les logs applicatifs et système en les centralisant et en limitant les accès.
Transmettre les JWT uniquement via HTTPS (TLS obligatoire) entre les microservices.
Vérifier systématiquement la signature cryptographique de chaque JWT avant toute utilisation par les microservices.
Refuser tout token avec alg=none et restreindre les algorithmes autorisés (ex: RS256, ES256 uniquement).
Stocker les clés de signature JWT de manière sécurisée (ex: AWS KMS, HashiCorp Vault), jamais en clair dans le code.
Utiliser des tokens courts (expiration courte) et ne pas faire confiance au contenu du payload sans validation côté backend.
Vérifier les rôles et permissions côté backend, jamais seulement via les claims du JWT.
Mettre en place une rotation régulière des clés de signature JWT.
Valider strictement toutes les entrées utilisateur utilisées dans les requêtes LDAP.
Utiliser des API ou fonctions sécurisées pour construire les requêtes LDAP, en évitant la concaténation de chaînes.
Limiter les filtres LDAP aux formats attendus via une allowlist.
Utiliser LDAPS (LDAP sécurisé via TLS) pour toutes les communications avec Active Directory et interdire les connexions LDAP non chiffrées.

MITIGATION À APPLIQUER
Appliquer le principe du moindre privilège pour le compte de service LDAP utilisé par l'application, en restreignant ses requêtes aux données strictement nécessaires et en interdisant l'accès aux attributs critiques (mots de passe, rôles).
Journaliser toutes les requêtes LDAP et centraliser les logs dans un SIEM pour détecter les tentatives d'injection.
Sanitiser le contenu des documents internes avant indexation et avant injection dans le contexte RAG : suppression de patterns d’injection, filtrage de markup suspect, décodage d’encodages ambigus.
N’autoriser l’ingestion des documents que depuis des sources approuvées avec traçabilité de provenance.
Mettre en place une "memory admission policy" stricte pour la base vectorielle pgvector, exigeant une AuthN/AuthZ forte sur chaque opération de lecture/écriture mémoire.
Appliquer des politiques de sécurité (ex: RBAC) aux opérations de lecture/écriture en mémoire.
Chiffrer les embeddings et la base de connaissances au repos (pgvector sur AWS RDS) et en transit.
Versionner les documents, embeddings et transformations pour permettre comparaison, détection de drift et rollback.
Surveiller les sorties et décisions dérivées du retrieval RAG pour détecter un chunk empoisonné qui ferait dévier l’agent.
Forcer l’utilisation de HTTPS (TLS 1.3 minimum) pour toutes les communications entre le frontend et le backend, et pour les appels API vers Azure OpenAI.
Implémenter mTLS (mutual TLS) pour l'authentification et le chiffrement de toutes les communications REST internes entre microservices.
Utiliser LDAPS (LDAP sécurisé via TLS) pour la communication avec Active Directory.
Valider strictement les certificats côté client et serveur pour toutes les communications sécurisées.
Désactiver les protocoles cryptographiques faibles (SSL, TLS 1.0, TLS 1.1) et les suites de chiffrement obsolètes.
Chiffrer les données critiques avant transmission.
Implémenter le certificate pinning côté client pour les communications critiques (ex: vers Azure OpenAI) si l'environnement le permet.

MITIGATION À APPLIQUER
Valider strictement chaque entrée (prompt) avant inférence avec le modèle Azure OpenAI, en rejetant les entrées mal formées, incomplètes ou hors distribution attendue.
Implémenter des AI Firewalls ou des guardrails en entrée pour détecter et filtrer les prompts adversariaux ou malveillants.
Réduire l'effet des perturbations mineures sur les entrées via des techniques de robustesse du modèle (si applicable via Azure OpenAI ou post-traitement).
Mettre en place une seconde validation (ex: un LLM secondaire ou des règles sémantiques) pour les décisions critiques du modèle.
Limiter les informations retournées par l'API du modèle au strict nécessaire, évitant d'exposer des scores ou probabilités qui pourraient aider un attaquant à affiner ses attaques.
Journaliser toutes les requêtes d'inférence et les réponses du modèle, ainsi que les décisions de rejet/blocage.
Déployer une API Gateway pour appliquer des quotas, du throttling et un rejet précoce des requêtes excessives.
Déployer un WAF devant les APIs exposées de l'application pour filtrer les floods applicatifs.
Implémenter un rate limiting strict par utilisateur, IP, clé API et tenant.
Limiter la taille maximale des prompts, des documents traités par RAG et le nombre maximal de tokens générés en sortie.
Définir des timeouts stricts pour l'inférence LLM et les appels à la base vectorielle.
Implémenter un autoscaling borné des microservices d'inférence RAG et un mode dégradé en cas de surcharge (ex: limiter le contexte RAG).
Monitorer le nombre de requêtes, la latence, les erreurs et la consommation CPU/mémoire/coût, et déclencher des alertes sur les seuils anormaux.
Restreindre l'accès réseau aux bases de données pgvector (AWS RDS) et aux services de génération d'embeddings aux seuls systèmes autorisés via des Security Groups et IAM stricts.
Appliquer un contrôle d'accès strict (RBAC) et le principe du moindre privilège pour les accès aux embeddings, pipelines RAG et artefacts.

MITIGATION À APPLIQUER
Limiter les informations retournées par les APIs d'inférence RAG au strict nécessaire (ex: ne pas exposer les scores de similarité bruts).
Implémenter une perturbation contrôlée des sorties du modèle si applicable pour empêcher la reconstruction (bien que plus complexe pour RAG).
Journaliser tous les accès aux modèles et registres ML, et surveiller les opérations de copie ou téléchargement des embeddings.
Déployer un Model Registry sécurisé pour gérer l'inventaire et la traçabilité des modèles et embeddings.
Appliquer une signature numérique obligatoire sur les documents et/ou les embeddings avant leur intégration dans pgvector et vérifier l'intégrité via hash cryptographique (SHA-256) avant mise en production.
Restreindre l'accès réseau aux bases de données pgvector et aux pipelines de génération d'embeddings aux seuls systèmes et rôles autorisés.
Implémenter un IAM strict et un RBAC granulaire sur les opérations d'ingestion, d'update et de déploiement des documents/embeddings.
Chiffrer les embeddings et les documents sources au repos et en transit.
Implémenter un versioning strict des documents et embeddings via un Model Registry (ou un système de versioning de données) et maintenir des copies immuables.
Détecter automatiquement les dérives statistiques (drift) dans les outputs du modèle RAG ou les distributions d'embeddings.
Implémenter un mécanisme de rollback rapide vers une version saine des documents ou embeddings.
Journaliser toutes les opérations sur les documents/embeddings (ingestion, update, rollback, deploy) et tous les accès à la base de connaissances.
Générer des tokens de réinitialisation longs, aléatoires, imprévisibles et à usage unique, avec une courte durée de validité.
Associer chaque token à un utilisateur unique côté serveur et invalider le token après première utilisation ou expiration.
Implémenter un flux de reset strict avec des étapes obligatoires et empêcher le contournement des étapes.

MITIGATION À APPLIQUER
Ne jamais exposer les tokens de reset dans les réponses API ou les logs.
Exiger une vérification supplémentaire (MFA) pour les comptes sensibles lors de la réinitialisation.
Limiter le nombre de demandes de reset par utilisateur et IP (rate limiting).
Notifier l’utilisateur par un canal de confiance (email/SMS) lors d’une demande de réinitialisation et après un changement de mot de passe.
Utiliser exclusivement HTTPS pour le transport des tokens de réinitialisation.
Utiliser des transactions atomiques pour toutes les opérations critiques sur les bases de données PostgreSQL.
Implémenter des mécanismes de versioning des données (optimistic locking) ou des verrous distribués si nécessaire.
Centraliser les opérations sensibles dans un service unique ou utiliser des files de traitement (queue) pour sérialiser les opérations critiques.
Empêcher les doubles insertions ou modifications (ex: s'assurer de l'idempotence des opérations).
Associer chaque action critique à un utilisateur authentifié et exiger une confirmation ou verrouillage pour ces opérations.
Journaliser toutes les opérations critiques (création, modification) avec des horodatages précis pour permettre l'audit.
Bloquer tout trafic sortant vers Internet si non nécessaire (egress deny) depuis les microservices.
Implémenter un firewall interne (Security Groups AWS) pour bloquer les appels au localhost et au réseau interne critique depuis les microservices.
Implémenter une allowlist stricte des URLs et des domaines autorisés pour les requêtes sortantes des microservices (y compris vers Azure OpenAI).
Ne jamais permettre à l’utilisateur de contrôler directement une URL dans les requêtes qui déclenchent des appels externes ou internes.
Bloquer l’accès à 169.254.169.254 (endpoints de métadonnées AWS) au niveau réseau ou via un proxy configuré.

MITIGATION À APPLIQUER
Appliquer le principe du moindre privilège sur les rôles IAM attachés aux instances des microservices.
Régénérer obligatoirement le session ID après chaque authentification réussie.
Utiliser exclusivement HTTPS pour toutes les sessions et interdire la transmission de session ID via URL (stocker uniquement dans des cookies sécurisés).
Activer les flags de sécurité sur les cookies (HttpOnly, Secure, SameSite=Strict).
Associer chaque session à un utilisateur authentifié unique et invalider les sessions après déconnexion, changement de mot de passe, ou détection d'anomalie.
Implémenter une durée maximale de session courte.
Journaliser tout le processus de gestion des sessions (création, régénération, invalidation).
Segmenter physiquement les buffers mémoire en zones d'instructions (L0), de travail (L1) et de données externes (L2) pour le contexte du LLM.
Appliquer un schéma de validation strict (JSON Schema) sur toute sortie d'outil (si des outils étaient impliqués) avant son intégration en mémoire de travail.
Utiliser un modèle tiers (Critique) pour valider que le résumé de conversation est sémantiquement fidèle aux logs bruts et exempt d'instructions malveillantes.
Ré-injecter systématiquement les règles de sécurité au sommet du contexte LLM après chaque phase de compression (Hard-Pinning).
Extraire des faits structurés au lieu de résumés narratifs pour limiter la ré-interprétation malveillante des instructions.
Imposer une authentification forte (MFA phishing-resistant, ex: FIDO2/WebAuthn) pour l'accès à l'application.
Exiger une step-up authentication avant toute action sensible initiée via le chatbot.
Utiliser des cookies HttpOnly + Secure + SameSite=Strict, des sessions courtes et régénérer l’ID de session après login.
Invalider immédiatement la session au logout, changement de mot de passe, ou suspicion de compromission.

MITIGATION À APPLIQUER
Ne jamais dériver l’identité de l’utilisateur depuis le prompt, la mémoire RAG, ou un output du modèle LLM ; résoudre l’identité uniquement depuis un jeton/session signé par l'IdP (Active Directory).
Vérifier les rôles et permissions dans un policy engine côté serveur, jamais via des informations déclarées dans le prompt par l'utilisateur.
Stocker les tokens d’authentification dans une session serveur sécurisée, jamais dans le prompt, la mémoire ou les logs du chatbot.